动手测试 Workshop：FastAPI Cars CRUD

通过三个测试层次建立信心：单元、集成和端到端（E2E）。此外还有一个
独立的 API 测试练习。API 测试属于集成层次，但单独成节，
因为 HTTP 行为值得专门学习。

对于每一节，你将：

1. 把一个完整测试复制到新文件中。
2. 运行它并看到通过的结果。
3. 故意改动一个断言，让测试失败。
4. 恢复断言，然后根据规格说明自己写一个新测试。

你无需修改应用即可完成这些练习。项目中已经包含额外的
示例测试，因此你的新文件可以和它们放在一起。

1. 搭建并运行应用

前提条件

- Python 3.11 或更高版本。
- 一个终端和一个文本编辑器（VS Code 即可）。
- 首次安装 Python 包和 Playwright 的 Chromium 浏览器时需要联网。

解压 cars-crud.zip，并在解压出的 cars-crud/ 目录中打开终端，
requirements-dev.txt 和 pytest.ini 就在该目录中。

macOS / Linux：

```
cd cars-crud
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
```

Windows PowerShell（这些命令无需激活虚拟环境）：

```
cd cars-crud
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
```

对于其余命令，在 macOS/Linux 上激活虚拟环境后使用 python。在
Windows 上，如果虚拟环境未激活，则用 .\.venv\Scripts\python.exe 代替
python。

启动应用

在终端 A 中、从 cars-crud/ 目录运行：

```
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

打开：

- 应用：http://127.0.0.1:8000
- 创建车辆页面：http://127.0.0.1:8000/cars/new
- 交互式 API 文档：http://127.0.0.1:8000/docs

手动创建一辆车、查看、编辑并删除它。这样可以在自动化之前熟悉预期
行为。SQLite 会自动初始化：应用把数据存在 cars.db 中。

探索应用时让终端 A 保持运行。打开一个新终端：终端 B，在同一个项目
目录中用于测试命令。在 macOS/Linux 上，用 source
.venv/bin/activate 激活 .venv（每个终端都要单独激活）。在 Windows 上，用 .\.venv\Scripts\python.exe
代替 python，如上所述。你可以用 Ctrl+C 停止应用。（自动化测试
不需要终端 A 保持运行。）

了解相关文件

```
cars-crud/
|-- app/
|   |-- schemas.py            # Pydantic 输入校验（单元测试目标）
|   |-- database.py           # SQLite 操作（集成测试目标）
|   |-- main.py               # FastAPI 页面与 API 路由
|   `-- static/               # 原生 HTML、CSS 和 JavaScript（E2E 目标）
|-- tests/
|   |-- conftest.py           # database_path、client、car_payload fixtures
|   |-- unit/
|   |-- integration/
|   |-- api/
|   `-- e2e/
|       `-- conftest.py       # page 和 live_server fixtures
|-- requirements-dev.txt
|-- pytest.ini
`-- TESTING_WORKSHOP.md
```

重要：每个测试都有自己的临时 SQLite 数据库。测试不会修改真实的 cars.db。
E2E 测试会自动启动它们自己的临时 FastAPI 服务器，因此你不必为它们
单独启动服务器。

2. 单元测试：不依赖数据库测试校验逻辑

目标：隔离地验证一小段 Python 代码。我们的目标是 app/schemas.py 中的
CarInput schema。此测试不打开 SQLite，也不发送 HTTP 请求。

步骤 A - 复制一个可用的测试

创建 tests/unit/test_workshop_unit.py 并粘贴：

```
from app.schemas import CarInput


def test_car_text_fields_are_trimmed():
    car = CarInput(
        make="  Mazda  ",
        model="  MX-5  ",
        year=2020,
        color="  Red  ",
    )

    assert car.make == "Mazda"
    assert car.model == "MX-5"
    assert car.color == "Red"
```

步骤 B - 运行它

```
python -m pytest tests/unit/test_workshop_unit.py -v
```

预期：1 passed。它检查 Pydantic 是否会去除输入字符串中的空白。

步骤 C - 故意让它失败

只修改这一行：

```
assert car.make == "Honda"  # 故意写错
```

再次运行同一条命令。预期：1 failed，带 AssertionError（实际值 Mazda，期望值 Honda）。把它改回“Mazda”，确认测试再次通过。

步骤 D - 写一个新的单元测试（你的任务）

在同一个文件中创建第二个测试，名为 test_blank_model_is_rejected。

要求：

- 尝试构建 CarInput(make="Mazda", model=" ", year=2020)。
- 断言会抛出 pydantic.ValidationError。
- 不要使用数据库或 HTTP 请求。
- 你的测试应能针对未修改的应用通过。

提示：import pytest，from pydantic import ValidationError，并使用 `pytest.raises(ValidationError): ...`。

再次运行该文件；现在你应该看到 2 passed。

3. 集成测试：用真实 SQLite 测试 Python 代码

目标：验证应用代码真的能在数据库中保存和读取数据。内置的 database_path fixture 为该测试提供一个全新的临时 SQLite 文件。

步骤 A - 复制一个可用的测试

创建 tests/integration/test_workshop_integration.py 并粘贴：

```
from app import database
from app.schemas import CarInput


def test_saved_car_can_be_read_from_sqlite(database_path):
    database.initialize(database_path)

    saved = database.create_car(
        database_path,
        CarInput(make="Toyota", model="Yaris", year=2020, color="Blue"),
    )
    loaded = database.get_car(database_path, saved.id)

    assert loaded is not None
    assert loaded.id == saved.id
    assert loaded.make == "Toyota"
    assert loaded.year == 2020
```

步骤 B - 运行它

```
python -m pytest tests/integration/test_workshop_integration.py -v
```

预期：1 passed。create_car() 和 get_car() 使用独立的 SQLite 连接，因此测试还检查了写入是否已提交并持久化。

步骤 C - 故意让它失败

修改：

```
assert loaded.year == 2021  # 故意写错
```

再次运行该测试。预期：1 failed（存储的年份是 2020）。恢复 2020 并重跑以看到通过。

步骤 D - 写一个新的集成测试（你的任务）

在同一个文件中创建 test_deleting_one_car_keeps_the_other。

要求：

- 使用 database.initialize(database_path) 初始化一个全新的 SQLite 数据库。
- 使用 database.create_car(...) 和 CarInput(...) 创建两辆不同的车。
- 使用 database.delete_car(database_path, first.id) 只删除第一辆车。
- 断言被删除的车无法用 database.get_car(...) 找到。
- 断言第二辆车仍然存在，且 database.list_cars(database_path) 中正好包含那一辆车。
- 你的测试应能针对未修改的应用通过。

提示：现有的 test_saved_car_can_be_read_from_sqlite 展示了如何创建车辆。
数据库函数在 app/database.py 中。

再次运行该文件；你应该看到 2 passed。

4. API 测试：测试 HTTP 契约

API 测试是一种集成测试。这里它们用 TestClient 针对 FastAPI 应用发起请求，无需启动 Uvicorn 或打开浏览器。client fixture 创建一个真实的、隔离的 SQLite 数据库。

步骤 A - 复制一个可用的测试

创建 tests/api/test_workshop_api.py 并粘贴：

```
def test_post_car_returns_201_and_json(client):
    payload = {
        "make": "Ford",
        "model": "Fiesta",
        "year": 2018,
        "color": "Black",
    }

    response = client.post("/api/cars", json=payload)

    assert response.status_code == 201
    car = response.json()
    assert car["id"] > 0
    assert car["make"] == "Ford"
    assert car["model"] == "Fiesta"
    assert response.headers["location"] == f"/api/cars/{car['id']}"
```

步骤 B - 运行它

```
python -m pytest tests/api/test_workshop_api.py -v
```

预期：1 passed。你验证了 HTTP 状态码、JSON 响应、生成的 ID 以及 Location 响应头。

步骤 C - 故意让它失败

修改：

```
assert response.status_code == 200  # 故意写错
```

重跑。预期：1 failed（API 正确返回 201 Created）。恢复 201。

步骤 D - 写一个新的 API 测试（你的任务）

在同一个文件中创建 test_invalid_year_is_not_saved。

要求：

- 发送 POST /api/cars，year 为 1800，其余车辆数据有效。
- 断言响应状态码为 422。
- 随后发送 GET /api/cars，断言它返回空列表。
- 不要使用浏览器或直接调用数据库函数。

提示：你可以复用 client.post(..., json=...) 和 client.get(...)。

再次运行该文件；你应该看到 2 passed。

5. 端到端（E2E）测试：在真实浏览器中测试

目标：测试完整链路：浏览器 -> HTML 表单 -> JavaScript fetch() -> FastAPI -> SQLite -> 浏览器显示。

步骤 A - 一次性安装 Chromium

```
python -m playwright install chromium
```

在 Linux 上，如果缺少系统库，使用文档中的 Playwright 命令安装这些依赖：

```
python -m playwright install --with-deps chromium
```

无需 pytest-playwright 插件；项目已经在 tests/e2e/conftest.py 中提供了 page 和 live_server fixture。

步骤 B - 复制一个可用的测试

创建 tests/e2e/test_workshop_e2e.py 并粘贴：

```
import re

from playwright.sync_api import expect


def test_user_creates_car_in_browser(page, live_server):
    page.goto(f"{live_server}/cars/new")

    page.get_by_label("Make", exact=True).fill("Toyota")
    page.get_by_label("Model", exact=True).fill("Yaris")
    page.get_by_label("Year", exact=True).fill("2020")
    page.get_by_label("Color", exact=False).fill("Blue")
    page.get_by_role("button", name="Create car").click()

    expect(page).to_have_url(re.compile(r"/cars/\d+$"))
    expect(page.get_by_role("heading", name="Toyota
Yaris")).to_be_visible()
```

步骤 C - 运行它

```
python -m pytest tests/e2e/test_workshop_e2e.py -v
```

在本地、Chromium 正常工作时预期：1 passed。你可以用如下命令观察浏览器：

```
python -m pytest tests/e2e/test_workshop_e2e.py -v --headed
```

无需手动启动 Uvicorn：live_server fixture 会为每个测试在空闲的
本地端口上启动和停止服务器。

步骤 D - 故意让它失败

把最后一个断言改成：

```
expect(page.get_by_role("heading", name="Toyota Corolla")).to_be_visible()
```

重跑。预期：1 failed，因为屏幕显示的是 Toyota Yaris，而不是 Toyota Corolla。Playwright
会等待错误的标题直到默认断言超时，然后报告失败。恢复 Toyota
Yaris。

步骤 E - 写一个新的 E2E 测试（你的任务）

在同一个文件中创建 test_user_edits_car_and_change_survives_reload。

要求：

- 打开 /cars/new，创建一辆 Honda Civic，年份 2022。
- 创建后，点击 Edit car 链接。
- 把 model 改成 Accord，year 改成 2023。
- 点击 Save changes。
- 断言详情页的标题是 Honda Accord。
- 刷新页面，断言 model 仍显示 Accord，year 为 2023。
- 只通过 Playwright 的浏览器操作交互；不要调用 Python 数据库函数或 TestClient。

提示：使用 page.get_by_role("link", name="Edit car")、page.get_by_label(...)、page.reload() 以及 expect(...)。详情页用 #car-model 和 #car-year 来显示数值。如需示例，可查看 tests/e2e/test_browser.py。

再次运行该文件；在 Chromium 能访问本地服务器的机器上，你应该看到 2 passed。

6. 运行所有测试并检查失败

完成练习后：

```
# 仅你的四个 workshop 文件（预期：8 个测试）
python -m pytest \
  tests/unit/test_workshop_unit.py \
  tests/integration/test_workshop_integration.py \
  tests/api/test_workshop_api.py \
  tests/e2e/test_workshop_e2e.py -v

# 全部测试（已有示例加上你的新练习）
python -m pytest -q
```

上面的多行命令适用于 macOS/Linux shell。在 Windows PowerShell 上，用一行：

```
python -m pytest tests/unit/test_workshop_unit.py
tests/integration/test_workshop_integration.py
tests/api/test_workshop_api.py tests/e2e/test_workshop_e2e.py -v
```

分别运行测试：

```
python -m pytest tests/unit -q
python -m pytest tests/integration -q
python -m pytest tests/api -q
python -m pytest tests/e2e -q
```

如何解读结果：

- PASSED：代码满足了测试的断言。
- FAILED：测试运行了，但某个断言不匹配；检查实际值与期望值。

- ERROR：测试无法搭建或运行（例如缺少浏览器库、导入错误或服务器启动问题）。

当测试失败时，先确定你在测试哪一层，再阅读最后的断言/错误信息，而不是整个 traceback。在故意失败的步骤中，FAILED 正是期望的观察结果。恢复断言后，同一个测试应回到 PASSED。

7. 完成检查清单

- [ ] 我能启动应用，手动创建、编辑并删除一辆车。
- [ ] 我复制的单元测试能通过、破坏断言后失败、恢复后再次通过。
- [ ] 我自己的单元测试会拒绝空的 model。
- [ ] 我复制的集成测试能通过、失败、再次通过。
- [ ] 我自己的集成测试删除一辆车而不影响另一辆。
- [ ] 我复制的 API 测试能通过、失败、再次通过。
- [ ] 我自己的 API 测试拒绝无效年份且不保存数据。
- [ ] 我复制的 E2E 测试能在真实浏览器中通过、失败、再次通过。
- [ ] 我自己的 E2E 测试编辑一辆车并验证改动在刷新后仍然存在。
- [ ] 我能解释每一层测试捕获什么问题，以及为什么测试不接触真实数据库。